

Tutorial Flutter Dasar untuk Pemula

Belajar Flutter dari setup sampai mini project dengan alur yang mudah diikuti: pahami konsep, jalankan contoh, lalu modifikasi sendiri.

Mulai belajar

Reset checklist

22

bagian materi

1

mini project

0

dependency web



flutter_ai_lab.dart

```
class LearningPath
```

```
  widgets.compose()
```

```
  state.update()
```

```
  navigator.push()
```

AI MENTOR

Next step

Build mini project catatan belajar.

```
api.fetch()
```



Panduan ini dibuat untuk developer yang sedang belajar Flutter dari nol sampai bisa membuat aplikasi sederhana. Fokusnya bukan hanya "copy paste jalan", tetapi memahami pola pikir Flutter: UI dibangun dari widget, data berubah lewat state, dan aplikasi berkembang dari komponen kecil yang dirangkai.

Daftar Isi

1. Apa itu Flutter
 2. Persiapan alat
 3. Membuat project pertama
 4. Struktur folder Flutter
 5. Dasar bahasa Dart
 6. Konsep widget
 7. Layout dasar
 8. StatelessWidget dan StatefulWidget
 9. State sederhana
 10. Navigasi antar halaman
 11. Input dan form
 12. List dan data dinamis
 13. Styling dan tema
 14. Mengambil data dari API
 15. Menyimpan data sederhana
 16. Mini project: aplikasi catatan belajar
 17. Debugging dan error umum
 18. Roadmap belajar berikutnya
-

1. Apa itu Flutter

Flutter adalah framework dari Google untuk membuat aplikasi mobile, web, dan desktop dari satu codebase. Flutter memakai bahasa Dart.

Kelebihan Flutter:

- Bisa membuat aplikasi Android dan iOS dari satu project.
- UI cepat dibuat karena semuanya berbasis widget.
- Ada hot reload untuk melihat perubahan dengan cepat.
- Cocok untuk aplikasi sederhana sampai aplikasi produksi.
- Dokumentasi dan komunitasnya besar.

Hal penting yang perlu dipahami sejak awal:

- Di Flutter, hampir semua bagian UI adalah widget.
 - Aplikasi dibuat dengan menyusun widget kecil menjadi tampilan besar.
 - Ketika data berubah, Flutter menggambar ulang bagian UI yang perlu berubah.
-

2. Persiapan Alat

Alat yang dibutuhkan:

- Flutter SDK
- Android Studio atau Visual Studio Code
- Emulator Android, simulator iOS, atau device fisik
- Git

Setelah instalasi Flutter, cek dengan command:

BASH

```
flutter doctor
```

Jika ada tanda silang, baca pesan yang muncul. Biasanya masalah awal ada di:

- Android SDK belum terpasang.
- Lisensi Android belum diterima.
- Emulator belum dibuat.
- Path Flutter belum masuk environment variable.

Untuk menerima lisensi Android:

BASH

```
flutter doctor --android-licenses
```

Untuk melihat device yang tersedia:

BASH

```
flutter devices
```

3. Membuat Project Pertama

Buat project Flutter baru:

BASH

```
flutter create belajar_flutter  
cd belajar_flutter  
flutter run
```

Jika memakai VS Code:

1. Buka folder project.
2. Pilih device di pojok kanan bawah.
3. Tekan **F5** atau jalankan **flutter run** dari terminal.

Saat aplikasi berjalan, coba ubah teks di file `lib/main.dart`, lalu tekan save. Flutter akan melakukan hot reload.

4. Struktur Folder Flutter

Struktur dasar project Flutter:

TEXT

```
belajar_flutter/  
  android/  
  ios/  
  lib/  
    main.dart  
  test/  
  pubspec.yaml
```

Penjelasan:

- `lib/` : tempat utama menulis kode Dart.
- `lib/main.dart` : entry point aplikasi.
- `android/` : konfigurasi native Android.
- `ios/` : konfigurasi native iOS.
- `test/` : tempat menulis test.
- `pubspec.yaml` : konfigurasi project, dependency, asset, font, dan versi aplikasi.

Untuk pemula, paling sering bekerja di:

- `lib/main.dart`
 - folder baru di dalam `lib/`, misalnya `pages/`, `widgets/`, `models/`, dan `services/`
 - `pubspec.yaml`
-

5. Dasar Bahasa Dart

Flutter menggunakan Dart. Berikut dasar yang perlu dikuasai.

Variable

DART

```
void main() {  
  String nama = 'Budi';  
  int umur = 20;  
  double tinggi = 170.5;  
  bool sudahLogin = true;  
  
  print(nama);  
  print(umur);  
  print(tinggi);  
  print(sudahLogin);  
}
```

var, final, dan const

DART

```
void main() {  
  var kota = 'Jakarta';  
  final tanggalLogin = DateTime.now();  
  const appName = 'Belajar Flutter';  
}
```

Bedanya:

- **var** : tipe data otomatis ditebak dari nilai awal.
- **final** : hanya bisa diisi sekali, nilainya bisa baru diketahui saat runtime.
- **const** : nilai tetap dan harus diketahui saat compile time.

Gunakan `final` untuk nilai yang tidak berubah setelah dibuat. Gunakan `const` untuk nilai yang benar-benar tetap.

Function

DART

```
int tambah(int a, int b) {  
    return a + b;  
}  
  
void sapa(String nama) {  
    print('Halo, $nama');  
}
```

Jika function hanya satu baris:

DART

```
int kali(int a, int b) => a * b;
```

List

DART

```
void main() {  
    final buah = ['Apel', 'Jeruk', 'Mangga'];  
  
    print(buah[0]);  
    print(buah.length);  
  
    for (final item in buah) {  
        print(item);  
    }  
}
```

Map

DART

```
void main() {  
  final user = {  
    'nama': 'Sari',  
    'email': 'sari@example.com',  
  };  
  
  print(user['nama']);  
}
```

Class

DART

```
class User {  
  final String nama;  
  final String email;  
  
  User({  
    required this.nama,  
    required this.email,  
  });  
}  
  
void main() {  
  final user = User(  
    nama: 'Rina',  
    email: 'rina@example.com',  
  );  
  
  print(user.nama);  
}
```

Null Safety

Dart mendukung null safety. Artinya, variable tidak boleh bernilai `null` kecuali kita izinkan.

DART

```
String nama = 'Andi';  
String? alamat;  
  
print(nama);  
print(alamat);
```

Tanda `?` berarti variable boleh `null` .

6. Konsep Widget

Widget adalah blok penyusun UI di Flutter. Contoh widget:

- `Text`
- `Container`
- `Column`
- `Row`
- `Image`
- `Scaffold`
- `AppBar`
- `ElevatedButton`
- `TextField`

Contoh aplikasi paling sederhana:

DART

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Halo Flutter'),
        ),
      ),
    );
  }
}
```

Penjelasan:

- `main()` adalah function pertama yang dijalankan.
 - `runApp()` menjalankan aplikasi Flutter.
 - `MaterialApp` memberi struktur aplikasi Material Design.
 - `Scaffold` memberi kerangka halaman seperti app bar, body, drawer, dan floating button.
 - `Center` membuat child berada di tengah.
 - `Text` menampilkan teks.
-

7. Layout Dasar

Layout Flutter dibuat dengan widget.

Column

`Column` menyusun widget dari atas ke bawah.

DART

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: const [  
    Text('Judul'),  
    Text('Deskripsi'),  
  ],  
)
```

Row

`Row` menyusun widget dari kiri ke kanan.

DART

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceBetween,  
  children: const [  
    Text('Kiri'),  
    Text('Kanan'),  
  ],  
)
```

Container

`Container` sering dipakai untuk memberi ukuran, padding, margin, warna, dan border.

DART

```
Container(  
  padding: const EdgeInsets.all(16),  
  margin: const EdgeInsets.all(12),  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(8),  
  ),  
  child: const Text(  
    'Kotak biru',  
    style: TextStyle(color: Colors.white),  
  ),  
)
```

Padding

DART

```
const Padding(  
  padding: EdgeInsets.all(16),  
  child: Text('Teks dengan jarak'),  
)
```

Expanded

Expanded membuat widget mengisi ruang kosong yang tersedia.

DART

```
Row(  
  children: [  
    Expanded(  
      child: Container(height: 80, color: Colors.red),  
    ),  
    Expanded(  
      child: Container(height: 80, color: Colors.green),  
    ),  
  ],  
)
```

8. StatelessWidget dan StatefulWidget

StatelessWidget

Gunakan `StatelessWidget` jika UI tidak punya data yang berubah dari dalam widget itu sendiri.

DART

```
class ProfileCard extends StatelessWidget {  
  const ProfileCard({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const Card(  
      child: Padding(  
        padding: EdgeInsets.all(16),  
        child: Text('Nama: Dinda'),  
      ),  
    );  
  }  
}
```

StatefulWidget

Gunakan `StatefulWidget` jika UI perlu berubah, misalnya counter, checkbox, form, tab, loading, atau data API.

DART

```
class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  void increment() {
    setState(() {
      count++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter')),
      body: Center(
        child: Text('Nilai: $count'),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: increment,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

`setState()` memberi tahu Flutter bahwa data berubah dan UI perlu digambar ulang.

9. State Sederhana

Contoh mengganti teks saat tombol ditekan:

DART

```
class GreetingPage extends StatefulWidget {
  const GreetingPage({super.key});

  @override
  State<GreetingPage> createState() => _GreetingPageState();
}

class _GreetingPageState extends State<GreetingPage> {
  String message = 'Selamat datang';

  void changeMessage() {
    setState(() {
      message = 'Kamu berhasil menekan tombol';
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('State Sederhana')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(message),
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: changeMessage,
              child: const Text('Ubah Teks'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Pola penting:

1. Simpan data sebagai variable di class `State` .
 2. Ubah data di dalam `setState()` .
 3. Tampilkan data di method `build()` .
-

10. Navigasi Antar Halaman

Buat halaman pertama:

DART

```
class HomePage extends StatelessWidget {  
  const HomePage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Home')),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            Navigator.push(  
              context,  
              MaterialPageRoute(  
                builder: (context) => const DetailPage(),  
              ),  
            );  
          },  
          child: const Text('Buka Detail'),  
        ),  
      ),  
    );  
  }  
}
```

Buat halaman kedua:

DART

```
class DetailPage extends StatelessWidget {
  const DetailPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Detail')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Kembali'),
        ),
      ),
    );
  }
}
```

Konsep:

- `Navigator.push()` membuka halaman baru.
 - `Navigator.pop()` kembali ke halaman sebelumnya.
-

11. Input dan Form

Untuk membaca input teks, gunakan `TextEditingController` .

DART

```
class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final emailController = TextEditingController();
  final passwordController = TextEditingController();

  @override
  void dispose() {
    emailController.dispose();
    passwordController.dispose();
    super.dispose();
  }

  void login() {
    final email = emailController.text;
    final password = passwordController.text;

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Login sebagai $email')),
    );

    print('Email: $email');
    print('Password: $password');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: emailController,
```

```

        decoration: const InputDecoration(
          labelText: 'Email',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 12),
      TextField(
        controller: passwordController,
        obscureText: true,
        decoration: const InputDecoration(
          labelText: 'Password',
          border: OutlineInputBorder(),
        ),
      ),
      const SizedBox(height: 16),
      SizedBox(
        width: double.infinity,
        child: ElevatedButton(
          onPressed: login,
          child: const Text('Login'),
        ),
      ),
    ],
  ),
);
}
}

```

Jangan lupa `dispose()` controller agar resource dibersihkan saat widget tidak dipakai lagi.

12. List dan Data Dinamis

Jika data sedikit, bisa pakai `Column`. Jika data banyak, gunakan `ListView.builder`.

DART

```
class ProductListPage extends StatelessWidget {
  const ProductListPage({super.key});

  @override
  Widget build(BuildContext context) {
    final products = [
      'Laptop',
      'Keyboard',
      'Mouse',
      'Monitor',
      'Headset',
    ];

    return Scaffold(
      appBar: AppBar(title: const Text('Produk')),
      body: ListView.builder(
        itemCount: products.length,
        itemBuilder: (context, index) {
          final product = products[index];

          return ListTile(
            leading: CircleAvatar(
              child: Text('${index + 1}'),
            ),
            title: Text(product),
            subtitle: const Text('Stok tersedia'),
            trailing: const Icon(Icons.chevron_right),
            onTap: () {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text('Pilih $product')),
              );
            },
          );
        },
      ),
    );
  }
}
```

Gunakan `ListView.builder` karena item dibuat sesuai kebutuhan, sehingga lebih efisien untuk data panjang.

13. Styling dan Tema

Daripada mengatur warna satu per satu di banyak tempat, gunakan theme.

DART

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      title: 'Belajar Flutter',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),  
        useMaterial3: true,  
        appBarTheme: const AppBarTheme(  
          centerTitle: true,  
        ),  
        inputDecorationTheme: const InputDecorationTheme(  
          border: OutlineInputBorder(),  
        ),  
      ),  
      home: const HomePage(),  
    );  
  }  
}
```

Contoh styling teks:

DART

```
Text(  
  'Belajar Flutter',  
  style: Theme.of(context).textTheme.headlineMedium?.copyWith(  
    fontWeight: FontWeight.bold,  
  ),  
)
```

Tips:

- Gunakan `Theme.of(context)` agar style konsisten.
 - Hindari hardcode warna terlalu banyak.
 - Pakai `SizeBox` untuk jarak sederhana.
 - Pakai `Padding` untuk memberi ruang di sekitar konten.
-

14. Mengambil Data dari API

Tambahkan dependency `http` di `pubspec.yaml` :

YAML

```
dependencies:  
  flutter:  
    sdk: flutter  
  http: ^1.2.0
```

Lalu jalankan:

BASH

```
flutter pub get
```

Contoh model:

DART

```
class Post {  
  final int id;  
  final String title;  
  final String body;  
  
  Post({  
    required this.id,  
    required this.title,  
    required this.body,  
  });  
  
  factory Post.fromJson(Map<String, dynamic> json) {  
    return Post(  
      id: json['id'] as int,  
      title: json['title'] as String,  
      body: json['body'] as String,  
    );  
  }  
}
```

Contoh fetch API:

DART

```
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

class PostPage extends StatefulWidget {
  const PostPage({super.key});

  @override
  State<PostPage> createState() => _PostPageState();
}

class _PostPageState extends State<PostPage> {
  late Future<List<Post>> postsFuture;

  @override
  void initState() {
    super.initState();
    postsFuture = fetchPosts();
  }

  Future<List<Post>> fetchPosts() async {
    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    );

    if (response.statusCode != 200) {
      throw Exception('Gagal mengambil data');
    }

    final List<dynamic> jsonList = jsonDecode(response.body) as List<dynamic>;

    return jsonList
      .map((item) => Post.fromJson(item as Map<String, dynamic>))
      .toList();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```

appBar: AppBar(title: const Text('Post')),
body: FutureBuilder<List<Post>>(
  future: postsFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    }

    if (snapshot.hasError) {
      return Center(child: Text('Error: ${snapshot.error}'));
    }

    final posts = snapshot.data ?? [];

    return ListView.builder(
      itemCount: posts.length,
      itemBuilder: (context, index) {
        final post = posts[index];

        return ListTile(
          title: Text(post.title),
          subtitle: Text(post.body),
        );
      },
    );
  },
);

```

Konsep penting:

- `Future` mewakili data yang datang nanti.
 - `async` dan `await` dipakai untuk operasi asynchronous.
 - `FutureBuilder` membantu menampilkan UI berdasarkan status data.
-

15. Menyimpan Data Sederhana

Untuk menyimpan data kecil seperti token, theme, atau preference user, gunakan package `shared_preferences` .

Tambahkan dependency:

YAML

```
dependencies:  
  flutter:  
    sdk: flutter  
  shared_preferences: ^2.2.0
```

Simpan data:

DART

```
import 'package:shared_preferences/shared_preferences.dart';  
  
Future<void> saveName(String name) async {  
  final prefs = await SharedPreferences.getInstance();  
  await prefs.setString('name', name);  
}
```

Ambil data:

DART

```
import 'package:shared_preferences/shared_preferences.dart';  
  
Future<String?> getName() async {  
  final prefs = await SharedPreferences.getInstance();  
  return prefs.getString('name');  
}
```

Hapus data:

DART

```
import 'package:shared_preferences/shared_preferences.dart';

Future<void> removeName() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.remove('name');
}
```

Catatan:

- `shared_preferences` cocok untuk data kecil.
 - Jangan simpan password langsung di `shared_preferences`.
 - Untuk data sensitif, pelajari secure storage.
 - Untuk data kompleks, pelajari SQLite, Hive, Isar, atau database lain.
-

16. Mini Project: Aplikasi Catatan Belajar

Di bagian ini kita membuat aplikasi catatan sederhana. Fitur:

- Menampilkan daftar catatan.
- Menambahkan catatan.
- Menghapus catatan.
- State masih disimpan di memory, jadi data hilang saat aplikasi ditutup.

Ganti isi `lib/main.dart` dengan kode berikut:

DART

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Catatan Belajar',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
        useMaterial3: true,
      ),
      home: const NotePage(),
    );
  }
}

class Note {
  final String title;
  final String description;

  Note({
    required this.title,
    required this.description,
  });
}

class NotePage extends StatefulWidget {
  const NotePage({super.key});

  @override
  State<NotePage> createState() => _NotePageState();
}
```



```

class _NotePageState extends State<NotePage> {
  final notes = <Note>[
    Note(
      title: 'Belajar Widget',
      description: 'Pahami Text, Container, Row, dan Column.',
    ),
    Note(
      title: 'Belajar State',
      description: 'Latihan setState dengan counter sederhana.',
    ),
  ];

  void openAddNotePage() async {
    final result = await Navigator.push<Note>(
      context,
      MaterialPageRoute(
        builder: (context) => const AddNotePage(),
      ),
    );

    if (result == null) return;

    setState(() {
      notes.add(result);
    });
  }

  void deleteNote(int index) {
    final deletedNote = notes[index];

    setState(() {
      notes.removeAt(index);
    });

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('${deletedNote.title} dihapus')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(

```

```

        title: const Text('Catatan Belajar'),
      ),
      body: notes.isEmpty
        ? const Center(
            child: Text('Belum ada catatan'),
          )
        : ListView.separated(
            padding: const EdgeInsets.all(16),
            itemCount: notes.length,
            separatorBuilder: (context, index) => const SizedBox(height: 8),
            itemBuilder: (context, index) {
              final note = notes[index];

              return Card(
                child: ListTile(
                  title: Text(note.title),
                  subtitle: Text(note.description),
                  trailing: IconButton(
                    icon: const Icon(Icons.delete_outline),
                    onPressed: () => deleteNote(index),
                  ),
                ),
              );
            },
          ),
      floatingActionButton: FloatingActionButton(
        onPressed: openAddNotePage,
        child: const Icon(Icons.add),
      ),
    );
  }
}

class AddNotePage extends StatefulWidget {
  const AddNotePage({super.key});

  @override
  State<AddNotePage> createState() => _AddNotePageState();
}

class _AddNotePageState extends State<AddNotePage> {
  final titleController = TextEditingController();
  final descriptionController = TextEditingController();

```

```

@override
void dispose() {
  titleController.dispose();
  descriptionController.dispose();
  super.dispose();
}

void saveNote() {
  final title = titleController.text.trim();
  final description = descriptionController.text.trim();

  if (title.isEmpty || description.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Judul dan deskripsi wajib diisi')),
    );
    return;
  }

  final note = Note(
    title: title,
    description: description,
  );

  Navigator.pop(context, note);
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Tambah Catatan'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          TextField(
            controller: titleController,
            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),
          ),

```

```

    ),
    const SizedBox(height: 12),
    TextField(
      controller: descriptionController,
      minLines: 3,
      maxLines: 5,
      decoration: const InputDecoration(
        labelText: 'Deskripsi',
        border: OutlineInputBorder(),
      ),
    ),
  ),
  const SizedBox(height: 16),
  SizedBox(
    width: double.infinity,
    child: FilledButton(
      onPressed: saveNote,
      child: const Text('Simpan'),
    ),
  ),
),
],
),
),
);
}
}

```

Yang dipelajari dari mini project ini:

- Membuat model sederhana dengan class `Note` .
- Menampilkan list dengan `ListView.separated` .
- Menambah data dari halaman lain.
- Mengirim data balik dengan `Navigator.pop(context, data)` .
- Menghapus item dari list.
- Validasi input sederhana.
- Membersihkan controller dengan `dispose()` .

Latihan lanjutan:

1. Tambahkan fitur edit catatan.

2. Tambahkan konfirmasi sebelum menghapus catatan.
 3. Simpan catatan memakai `shared_preferences` .
 4. Pisahkan file menjadi `main.dart` , `note.dart` , `note_page.dart` , dan `add_note_page.dart` .
 5. Tambahkan pencarian catatan.
-

17. Debugging dan Error Umum

Error: No connected devices

Solusi:

- Jalankan emulator.
- Sambungkan device fisik.
- Aktifkan USB debugging di Android.
- Cek dengan `flutter devices`.

Error: dependency belum terinstall

Solusi:

BASH

```
flutter pub get
```

UI overflow kuning hitam

Biasanya karena widget terlalu besar untuk layar.

Solusi yang sering dipakai:

- Bungkus konten dengan `SingleChildScrollView`.
- Gunakan `Expanded` di dalam `Column` atau `Row`.
- Kurangi ukuran widget.
- Pastikan layout responsif.

Contoh:

DART

```
SingleChildScrollView(  
  child: Column(  
    children: [  
      Text('Konten panjang'),  
    ],  
  ),  
)
```

setState() called after dispose()

Biasanya terjadi ketika async process selesai setelah halaman ditutup.

Solusi:

DART

```
if (!mounted) return;  
  
setState(() {  
  // ubah state  
});
```

Lupa dispose controller

Jika memakai `TextEditingController`, biasakan:

DART

```
@override  
void dispose() {  
  controller.dispose();  
  super.dispose();  
}
```

Hot reload tidak mengubah state awal

Hot reload mempertahankan state. Jika ingin reset total, gunakan hot restart.

18. Roadmap Belajar Berikutnya

Setelah menguasai dasar di tutorial ini, lanjutkan ke topik berikut:

1. Dart lebih dalam

- null safety
- async await
- class dan inheritance
- extension
- collection method seperti `map`, `where`, dan `fold`

1. Flutter UI

- layout responsif
- custom widget
- theme
- dark mode
- animation dasar

1. State management

- `setState`
- `ValueNotifier`
- Provider
- Riverpod
- Bloc atau Cubit

1. Networking

- REST API
- JSON parsing
- error handling
- loading state

- retry dan timeout

1. Local storage

- shared preferences
- secure storage
- SQLite
- Hive atau Isar

1. Arsitektur aplikasi

- pemisahan folder
- repository pattern
- service layer
- dependency injection
- clean architecture dasar

1. Testing

- unit test
- widget test
- integration test

1. Publish aplikasi

- build APK
 - build App Bundle
 - konfigurasi icon
 - konfigurasi splash screen
 - upload ke Play Store
-

Rekomendasi Cara Belajar

Belajar Flutter akan lebih cepat jika dilakukan bertahap:

1. Ketik ulang kode, jangan hanya membaca.
2. Jalankan setiap contoh kecil.
3. Ubah teks, warna, layout, dan data supaya paham efeknya.
4. Buat project mini setelah belajar satu konsep.
5. Biasakan membaca error dari atas ke bawah.
6. Jangan langsung lompat ke state management kompleks sebelum nyaman dengan `setState`.

Urutan latihan yang disarankan:

1. Counter sederhana.
 2. Form login tanpa backend.
 3. List produk statis.
 4. Catatan sederhana.
 5. Fetch data dari API.
 6. Simpan data lokal.
 7. Aplikasi CRUD sederhana.
-

Checklist Penguasaan Dasar

Gunakan checklist ini untuk menilai progress:

☐ Bisa membuat project Flutter baru.	☐ Bisa menjalankan aplikasi di emulator atau device.
☐ Paham struktur folder dasar.	☐ Bisa membuat <code>StatelessWidget</code> .
☐ Bisa membuat <code>StatefulWidget</code> .	☐ Bisa memakai <code>setState</code> .
☐ Bisa membuat layout dengan <code>Column</code> , <code>Row</code> , <code>Container</code> , dan <code>Padding</code> .	☐ Bisa memakai <code>ListView.builder</code> .
☐ Bisa membaca input dari <code>TextField</code> .	☐ Bisa membuat navigasi antar halaman.
☐ Bisa membuat model class sederhana.	☐ Bisa mengambil data API dengan <code>http</code> .
☐ Bisa memakai <code>FutureBuilder</code> .	☐ Bisa menyimpan data sederhana.
☐ Bisa membaca dan memperbaiki error umum.	

Jika sebagian besar checklist sudah terpenuhi, kamu sudah punya fondasi yang cukup kuat untuk lanjut ke project nyata.

Penutup

Flutter terasa besar di awal karena banyak widget dan konsep baru. Namun pola dasarnya sederhana: buat widget, susun layout, simpan data sebagai state, ubah state saat user berinteraksi, lalu tampilkan hasilnya.

Mulailah dari aplikasi kecil. Setelah nyaman, tingkatkan perlahan dengan API, local storage, state management, dan arsitektur yang lebih rapi. Yang paling penting adalah sering membuat sesuatu, karena skill Flutter tumbuh dari latihan langsung.